

Examen POO – 3 septembrie 2019

1p din oficiu

1. (0,5p) Definiți conceptul de **variabilă de clasă** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA de definire a uneia și apoi de utilizare a ei.

2. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
public class A {  
    private byte a = -10;  
    private byte b = 100;  
    public static void main(String[] args) {  
        int c = a * b;  
        System.out.println(c);  
    }  
}
```

3. (0,5p) Definiți conceptul de **clasă abstractă** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA de definire a uneia și apoi de utilizare a ei.

4. (0,5p) Definiți conceptul de **ascundere a variabilelor în relația de moștenire** și dați apoi un exemplu simplu în JAVA.

5. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
interface B {  
    public static String x = "Examen";  
}
```

```
public class A implements B {  
    public void method() {  
        System.out.println(B.x);  
    }  
    public static void main(String[] args) {  
        B obj = new A();  
        obj.method();  
    }  
}
```

6. (0,5p) Definiți conceptul de **excepție neverificată** și dați apoi un exemplu simplu de aruncare explicită a unei astfel de excepții în JAVA.

7. (0,5p) Dați un exemplu în JAVA de folosire a unei **colecții** în care să rețineți un șir de numere reale care poate conține și duplicate. Colecția trebuie să fie eficientă la multiple operații de inserare și ștergere de elemente.

8. (0,5p) Definiți procesul de **serializare a unui obiect** și dați un exemplu simplu în JAVA.

9. (0,5p) Dați un exemplu simplu în JAVA pentru definirea și crearea unui **fir de lucru (thread)** și apoi pornirea execuției lui.

10. (1,5p) Ce se afișează la ieșirea standard în urma execuției următorului program JAVA:

<pre> class A { static int c; String x = "1"; public A() { x+="11"; c++; } public String myMethod(String t) { x+=t; return x; } public String getX() { return x; } } class B extends A { String x = "2"; public B() { x+="22"; c++; } public int myMethod(int t) { x+=t; return 555; } } </pre>	<pre> public class C { public static void main (String[] args) { A a = new A(); System.out.println(a.getX()); A b1 = new B(); B b2 = (B) b1; new B(); System.out.println(b1.x); System.out.println(b2.myMethod("8")); System.out.println(b2.myMethod(9)); System.out.println(b2.x); System.out.println(b2.getX()); System.out.println(b1==b2); System.out.println(a.equals(b1)); System.out.println(a.getClass(). getSuperclass().getName()); System.out.println(A.c); } } </pre>
--	---

11. (3p) Un feribot are disponibile două zone pentru transportul de autovehicule și are definită o masă maximă admisă cu care poate fi încărcat. Într-o zonă sunt transportate numai autoturisme iar în cealaltă zonă numai camioane. Numărul de locuri disponibil din cadrul celor două zone de transport diferă de la un feribot la altul. Orice autovehicul este caracterizat prin numele proprietarului și masa autovehiculului. Un autoturism are în plus specificat numărul de locuri iar un camion faptul dacă transportă sau nu mărfuri periculoase.

Respectând paradigma programării orientată pe obiecte, identificați entitățile, relațiile dintre acestea, desenați diagrama de clase UML detaliată și implementați în JAVA un program care să rezolve următoarele cerințe:

- Crearea de feriboturi (cu o anumită masă maximă admisă, o anumită capacitate a celor două zone de transport și încă neîncărcate)
- Crearea de autovehicule (autoturisme și camioane) prin citirea tuturor informațiilor de la intrarea standard
- Încărcarea unui autovehicul pe feribot. Se vor utiliza colecții pentru stocarea autovehiculelor în cele două zone ale feribotului. Dacă în momentul încărcării unui autovehicul nu mai există locuri disponibile în zona dedicată lui, sau se depășește masa maximă admisă a feribotului, se va arunca o excepție verificată, iar autovehiculul nu va fi încărcat pe feribot
- Determinarea numărului de camioane de pe feribot care transportă mărfuri periculoase
- Afișarea tuturor detaliilor despre toate autovehiculele aflate la un moment dat pe feribot în ordinea alfabetică a numelui proprietarilor
- Scrieți o metodă *main* în care creați mai întâi un feribot, apoi executați toate cerințele specificate mai sus astfel încât să utilizați fiecare funcționalitate cel puțin o dată